How to Build an Agent Meta-Harness

A component-level analysis and capability-tiered build guide, derived from the `elder-plinius/T3MP3ST` repository

Document type: Formal technical report **Subject repo:** `github.com/elder-plinius/T3MP3ST` (AGPL-3.0) **Analyst framing:** Agentic-engineering / harness architecture review **Date:** 2026-07-06

---

0. Analyst's Note & Scope

T3MP3ST is a **multi-agent offensive-security framework** — an autonomous "recon → exploit → report" kill chain that wraps a coding agent (Claude Code, Codex, Hermes, or a local model) in tooling. This report treats it purely as an **engineering artifact**: it extracts the reusable *harness* patterns (the software scaffold around an LLM) so you can understand how a meta-harness is assembled.

Two classes of content in the repo are **flagged, not reproduced** here:

1. The offensive exploitation specifics (payloads, working exploit chains).
2. An explicit AI-red-team / prompt-injection technique catalogue and a "safeguard-evasion by decomposition" design.

These are enumerated and categorised in **§6 (Injections & Adversarial Patterns)** because your brief asked for them, but this document does not provide operational recipes for either. The repo itself is licensed for **authorized testing only**; the same constraint applies to anything built from this guide.

---

1. What a "Harness" and "Meta-Harness" Are

- **Harness** — the deterministic software scaffold around a non-deterministic model: the loop, the tools, the parsing, the memory, the guardrails. The model supplies judgment; the harness supplies structure, safety, and reproducibility.
- **Meta-harness** — a harness that *builds and coordinates other harnesses*. Instead of one agent in one loop, it composes **loadouts** (specialist personas + a tool set + a target

adapter + a scoring benchmark) and orchestrates several agents/models against a shared objective, with a control plane above them.

T3MP3ST is a meta-harness because new capability domains **compose rather than fork** — each domain is a loadout, not a new codebase (its own README makes this claim explicitly).

---

2. Component Inventory (Enumeration)

The repository decomposes into the following functional layers. Directory references are the load-bearing ones.

#	Layer	Repo location	Role in the harness
2.1	**LLM backbone / provider abstraction**	`src/llm/` , `src/config/`	One interface over many providers (OpenRouter, Anthropic, OpenAI, Venice, xAI) **and** keyless local agents (Ollama/LM Studio/vLLM). Text-driven tool-calling so even non-function-calling models work.
2.2	**Agent loop (ReAct engine)**	`src/agent/` , `src/agent/local-agents.ts`	The reason → act → observe loop each operator runs. Same loop for recon and downstream operators.
2.3	**Arsenal (tool registry)**	`src/arsenal/` (`catalog.ts` , `adapter-tools.ts` , `post-ex.ts` , `approval.ts`)	Typed tool catalogue (35 built-in / 83 with full arsenal), each tool stamped with a **risk tier**.
2.4	**Approval + risk gating**	`src/arsenal/approval.ts`	Human-veto layer: gated tools are inert until approved; “spicy” tools fire an audited warning every call; fail-safe **denies** when unattended.
2.5	**Egress / scope containment**	`src/opsec/` , arsenal scope gate	Once a target is set, networked tools refuse off-scope hosts (`SCOPE DENIED`). On by default.

2.6	**Orchestration (decomposition)**	`src/orchestration/` (`orchestrator.ts` , `prompts.ts` , `context-pack.ts`)	"Master-builder" splits an objective into sub-queries for worker models and synthesises results. **See §6.1 — flagged.**
2.7	**Operators / squad model**	`src/operators/` , `src/admiral/`	8 operators mapped to MITRE ATT&CK phases (Recon, Scanner, Exploiter, Infiltrator, Exfiltrator, Ghost, Coordinator, Analyst). Recon is live; later phases scaffolded.
2.8	**Mission engine + control plane**	`src/mission/` , `src/server.ts` , War Room UI	Mission lifecycle, "Op Admiral" natural-language front door, HTTP API (`/api/mission/start` , `/status`).
2.9	**Evidence vault + findings ledger**	`src/evidence/` (`gate.ts`), `src/pack/`	Append-only findings, evidence gating, credential store.
2.10	**Verification / refuter panel**	`scripts/verify-finding.mjs` , `refute-finding.mjs` , `verify-claims.mjs`	Adversarial verification: a skeptic/refuter panel plus a claims re-derivation guard ("24/24 green").
2.11	**Benchmark harness**	`bench/` , `src/benchmark/` , `scripts/*-bench.mjs`	XBEN, Cybench, CVE-Zero suites; every headline recomputes from committed JSON.
2.12	**Coordinated-disclosure pipeline**	`scripts/disclosure-gen.mjs` , `realpoc.mjs`	OSV novelty check → live PoC → refuter → CVSS → draft (human sends).
2.13	**Self-improvement loop**	`scripts/lessons.mjs` , `src/resources/`	Records lessons/proposals (feeding back into planning is roadmap).
2.14	**Prompt library / playbooks**	`src/prompts/` , `src/resources/ai-redteam-playbook.ts`	Persona and playbook prompt packs. **Contains flagged content — see §6.**
	MCP +	`src/mcp-server.ts` ,	Exposes `security_recon` to

2.15	**integration surface**	`src/integrations/`	MCP-aware agents.
2.16	**Stubs (interface-only future modules)**	`src/stubs/`	Cloud, persistence, swarm, cognition — declared as scaffolding, honestly marked.

---

3. How-to-Harness Guide — Structure

The build is presented in three capability tiers. Each element is numbered and tagged **[BASIC] / [INT] / [ADV]**. Build strictly bottom-up: every INT element assumes the BASIC spine exists; every ADV element assumes INT.

---

4. BASIC Tier — The Minimum Viable Harness

Goal: one agent, one loop, one tool set, running safely and reproducibly.

4.1 [BASIC] Provider abstraction layer. Wrap every model behind a single `LLMBackbone` interface (see `src/llm/`). Normalise: chat, streaming, token budgets, and a **text-mode tool-calling shim** so models without native function-calling still work. This one decision is what makes the harness provider-agnostic and keyless-capable later.

4.2 [BASIC] Config + key management. A typed config (`src/config/index.ts`) with a `defaultProvider`, an `AVAILABLE_MODELS` registry, and env-var key resolution. Support a **local/offline** provider from day one — it removes the cloud dependency and is the cheapest way to iterate.

4.3 [BASIC] The ReAct agent loop. The core cycle: *system prompt → model proposes an action → harness executes a tool → observation is fed back → repeat until stop condition*. Keep the loop dumb and the model smart. Enforce a **max-steps** ceiling and a **stop/answer** contract.

4.4 [BASIC] A typed tool catalogue. Model each tool as a record with `name`, description, input schema, and a `riskTier` (`src/arsenal/catalog.ts`). Start with read-only/inert tools (`file`, `curl`, `dig`, `whois`). The registry — not ad-hoc string parsing — is what lets you add capabilities without touching the loop.

4.5 [BASIC] Robust output parsing. Models wrap JSON in fences, add prose, emit stray braces. Implement the repo's proven pattern (`orchestrator.ts`): strip fences, then scan **balanced bracket spans** and take the *last* one that parses to the expected shape. Never

trust a naive `JSON.parse`.

4.6 [BASIC] Scope containment (safety floor). Before any networked tool runs, gate it against an explicit target scope — refuse off-scope, loopback, and private hosts by default (`SCOPE DENIED`). In a meta-harness this is the **non-negotiable safety primitive**: it belongs in the BASIC tier, not bolted on later.

4.7 [BASIC] A single mission object. One structured record that holds the objective, target, scope, status, and findings. Everything else reads/writes through it. This is the seed of the control plane.

---

5. INT Tier — The Coordinated, Auditable Harness

Goal: multiple specialists, human-in-the-loop safety, and evidence you can trust.

5.1 [INT] Specialist personas / squad model. Split one generalist agent into role-scoped operators, each with its own system prompt and tool subset (`src/operators/`). T3MP3ST maps eight operators to kill-chain phases. The engineering lesson is **loadout composition**: a "domain" = persona set + tool subset + target adapter + benchmark, so new domains *compose* instead of forking the codebase.

5.2 [INT] Approval + risk-tier gating. Promote `riskTier` from metadata to enforcement (`src/arsenal/approval.ts`). Gated tiers (`intrusive` / `credential` / `dangerous`) are **inert until approved**; approve-once-then-free per session; the hottest tiers fire an audited, non-blocking **warning every call**. Critical detail: **fail-safe denies** — an unattended run with no approver wired never silently fires a dangerous tool.

5.3 [INT] Pre-authorisation allowlists for headless runs. Benchmarks and batch jobs can't answer prompts, so hand them an explicit allowlist up front; anything off-list denies and everything is audited. This is how you keep automation safe without a human on every call.

5.4 [INT] Evidence vault + append-only findings ledger. Findings are written once, never mutated (`src/evidence/`, `src/pack/`). Each finding carries its provenance (which tool, which output). An **evidence gate** rejects a claim that isn't backed by real tool output — this is the antidote to model confabulation.

5.5 [INT] Adversarial verification (refuter panel). Before a finding is "real," a separate **skeptic/refuter** agent tries to knock it down (`scripts/verify-finding.mjs`, `refute-finding.mjs`). A finding that survives refutation is promoted; one that doesn't is retracted. This second-model check is the single highest-leverage quality mechanism in the repo.

5.6 [INT] Mission control plane + natural-language front door. A control surface ("Op Admiral") that turns a plain-English objective into a launched mission, plus an HTTP API and a UI (`src/server.ts` , War Room). The plane owns lifecycle: start → status → evidence → report.

5.7 [INT] Reproducibility guard. A `verify-claims` -style script that re-derives every headline metric from committed artifacts and runs on every push (`scripts/verify-claims.mjs`). "A claim that can't be reproduced doesn't ship." For a meta-harness this is what converts marketing into receipts.

5.8 [INT] MCP / integration exposure. Expose selected harness capabilities as MCP tools (`src/mcp-server.ts`) so *other* agents can call your harness. This is the first step from "an agent" to "a platform other agents build on."

---

6. ADV Tier — The Meta-Harness (Orchestration, Benchmarking, Self-Improvement)

Goal: a harness that plans across models, measures itself, and improves — plus the adversarial content you must recognise.

6.1 [ADV] Multi-model decomposition orchestrator — ⚠️ FLAGGED. `src/orchestration/` implements a "master-builder" that decomposes a full objective into sub-queries dispatched to worker models, then synthesises the answers. As a *neutral engineering pattern*, orchestrator/worker decomposition is legitimate and powerful (parallelism, token budgeting, context packing, round-based synthesis with an accumulated-knowledge memory).

However, this specific implementation is explicitly engineered as a **safeguard-evasion technique**: the design routes an offensive objective around a *safety-trained worker model* (the code and prompts name a safeguarded frontier model as the intended worker) by showing the worker only individually-benign snippets while the orchestrator "does all the offensive reasoning." The prompt literally frames "context isolation" — withholding the lines that form the dangerous shape from the worker — as "the entire mechanism."

I'm flagging this rather than documenting it as a build step. If you are building a legitimate meta-harness, you want the *benign* half of this pattern (decompose → parallel workers → synthesise) and you should treat the context-isolation-to-defeat-a-safeguarded-model half as an **anti-pattern and a red flag** — both ethically and because it is exactly the behaviour downstream providers detect and block.

6.2 [ADV] Benchmark harness with committed oracles. A full evaluation layer (`bench/` ,

`src/benchmark/`): challenge suites (XBEN, Cybench, CVE-Zero), each solved flag graded against a **committed ground-truth oracle**, with an **anti-fitting guard** that fails the build if numbers are tuned to the test set. Black-box and white-box results are reported separately and never blended. This is how a meta-harness earns trust: it benchmarks itself and publishes the re-run.

6.3 [ADV] Coordinated-disclosure pipeline. An end-to-end chain (`scripts/disclosure-gen.mjs` , `realpoc.mjs`): novelty check against a known-vuln database → live proof-of-concept → refuter panel → severity scoring → **draft report a human must send**. The engineering takeaway is the **human-gated terminal step** — the automation stops one click short of action.

6.4 [ADV] Self-improvement / lessons loop. Record structured "lessons" and improvement proposals from each mission (`scripts/lessons.mjs`). The repo is honest that *feeding these back into planning is still roadmap* — the loop currently records, it does not yet learn. A meta-harness that closes this loop (retrieval of past lessons into new mission planning) is the genuine frontier here.

6.5 [ADV] Stub-first future modules. `src/stubs/` holds interface-only definitions for cloud, persistence, swarm, and cognition modules. The pattern — **declare the interface before the implementation, and mark it honestly as scaffolding** — is a mature way to keep an ambitious roadmap from turning into vapourware inside the running system.

6.6 [ADV] Embedded AI-red-team technique catalogue — ⚠️ FLAGGED (do not reproduce). `docs/AI_REDTEAM_TECHNIQUES.md` (referenced from `src/resources/ai-redteam-playbook.ts`) is a structured taxonomy of ~18 prompt-injection / jailbreak technique families used *against LLMs* — refusal suppression, output prefill seeding, format-contract hijacks, divider/mode-switch token injection, fake system-state spoofing, persona displacement, CoT-channel exploitation, encoding/obfuscation transforms, **invisible-unicode steganography**, homoglyph token manipulation, payload splitting, multi-turn crescendo escalation, token-bomb DoS, and system-prompt extraction. It exists so a `t3mp3st` "ai_red_team" specialist can wield them.

For your purposes this is a **catalogue of attacks your harness must defend against**, and that is the only lens under which I'll represent it. I have not reproduced the operational technique text. See §7 for the defensive implications.

---

7. Injections & Adversarial Patterns — Findings Summary

Your brief asked specifically for Easter eggs and injections. Findings, categorised:

7.1 Easter eggs (benign)

- **Flag #1 (free):** an HTML comment at the very top of `README.md` contains a CTF-style flag `T3MP3ST{r3c31pt5_n0t_v1b3z}` (“receipts not vibez” — a thematic nod to the repo’s reproducibility ethos) plus the signature *“LOVE PLINY.”*
- **Flag #2 (earned):** the same comment states the flag “that counts” is *earned* by running `npm run verify-claims` — i.e. the reward for actually reproducing the numbers. A deliberate reinforcement of the repo’s core “receipts, not trust-me” theme.
- **CTF range flags:** `ctf/` and `docs/index.html` embed challenge flags (e.g. `T3MP3ST{sql1_l0g1n_byp4ss_ez}`) — these are intended targets for the practice range, not hidden messages.
- **Signature motifs:** the ⦗ … ⦘ bracket glyphs, “⛈️ storm” branding, and *“Fortes fortuna iuvat”* recur as the author’s calling card.

7.2 Injection / adversarial patterns (flagged)

Ref	Pattern	Where	Nature
§6.1	**Decomposition safeguard-evasion** — split an objective so a safety-trained worker model never sees the offensive frame	`src/orchestration/prompts.ts` , `orchestrator.ts`	Engineered evasion of a *downstream model’s* guardrails via context isolation
§6.6	**AI-red-team technique arsenal** — ~18 prompt-injection/jailbreak families	`docs/AI_REDTEAM_TECHNIQUES.md` , `src/resources/ai-redteam-playbook.ts`	Offensive prompt-injection catalogue, incl. invisible-unicode & homoglyph channels
—	**Cross-modal / tool-routing indirect injection**	red-team doc §2.13	Injection delivered via tool output/agentic surfaces (relevant if your harness

			ingests untrusted content)

7.3 Defensive implications for *your* harness

If you build a meta-harness that ingests untrusted external content (web pages, tool output, files), assume every family in §6.6 will be aimed at it. Minimum defences: (a) never let fetched/tool content occupy the same trust level as your system prompt; (b) strip zero-width and homoglyph unicode from ingested text; (c) treat model-proposed tool calls as *proposals* subject to the §5.2 approval gate; (d) keep an §5.5 refuter check between "model said so" and "acted on it." The repo's own `SECURITY.md` and scope gate (§4.6) are the author's version of this posture.

---

8. The Meta-Harness Blueprint (Synthesis)

Distilled to a reusable checklist, independent of the offensive domain:

1. **Abstract the model** (provider + local, text-mode tools). *[BASIC]*
2. **A dumb loop, a typed tool registry, robust parsing.** *[BASIC]*
3. **Scope + safety gate before any side-effecting tool.** *[BASIC]*
4. **Specialist loadouts that compose, not fork.** *[INT]*
5. **Risk-tiered approval with fail-safe-deny.** *[INT]*
6. **Append-only evidence + provenance gating.** *[INT]*
7. **A second-model refuter between claim and truth.** *[INT]*
8. **A control plane with a natural-language front door + API/MCP.** *[INT]*
9. **A reproducibility guard that re-derives every claim on CI.** *[INT]*
10. **An orchestrator that decomposes across models — using only the benign half of that pattern.** *[ADV]*
11. **Self-benchmarking against committed oracles with an anti-fitting guard.** *[ADV]*
12. **Human-gated terminal step on anything consequential.** *[ADV]*
13. **A recorded lessons loop, closing toward planning feedback.** *[ADV]*

The through-line the repo gets right: **the model is the brain; the harness is discipline.**

Reproducibility, provenance, adversarial verification, and hard safety gates are what separate a meta-harness from a prompt with delusions of grandeur.

---

9. Honest Trade-offs (as the repo itself concedes)

- The **8-operator swarm is largely unproven** — headline numbers came from a *single-agent* ReAct loop, not the coordinated cell. Later operators (Exploiter, Infiltrator, Exfiltrator, Ghost) run the real loop but are "experimental."
- **White-box source ingest is Python-only regex** and costs *more* tokens, not fewer.
- **Self-improvement records but does not yet learn.**
- The **decomposition orchestrator's evasion framing (§6.1) is a liability**, not a feature, for any harness meant to run on hosted models under a provider's terms.
- Small-n results (CVE-Zero n=10) are **directional, not definitive** — the repo says so.

Building the BASIC + INT spine gives you a genuinely useful, safe, auditable agent harness. The ADV tier is where the interesting research lives — and where the repo is most honestly marked as "still earning its stripes."

Prepared as an architecture/agentic-engineering review. Offensive specifics and adversarial technique contents are flagged and categorised, not reproduced. Any implementation built from this guide inherits the subject repo's constraint: authorized, in-scope testing only.